search feature is that it's fast and it handles the edge cases that make coding a binary search hard. The tricky thing is that the setup might not be what you expect.

In order for this feature to work, the array has to be sorted, but exactly what Ruby means by that is a little different from just a numerical sort.

The bsearch method takes a block. The block takes an element and returns a value. One of the two cases must be true of the block and the array:

- The block returns true or false. All elements of the array for which the block returns false must come before any element of the array for which the block returns true. In this case, bsearch returns the first element of the array for which the block returns true, or nil if there aren't any such elements.
- The block returns a numeric value. The numeric values don't have to be in order, but all the elements for which the block returns positive must come first, followed by all elements for which the block returns zero, followed by all elements for which the block returns negative. In this case, bsearch will return an element for which the block returns zero, though if we are reading the docs correctly, it is not guaranteed to be the first such element. Again, it returns nil if there are no elements for which the block returns zero.

Note that for speed purposes, bsearch does not check that the ordering matches the constraints above (because the time to do so would defeat the purpose of the bsearch method). It's on you to guarantee the precondition.

The related method bsearch_index does the same logic but returns the index of the result rather than the value:

```
sample = (1 .. 10000).to_a
sample.bsearch { |i| i  >= 512 }  # => 512
sample.bsearch { |i| 512 <=> i }  # => 512
```

The first search is an example of the first condition; the second search is an example of the second condition.

Packing and Unpacking

Ruby has a mechanism for converting arrays into binary strings and back again, which can be useful for compact custom encoding of data, or for decoding known binary data.

On the array side, the pack method takes a template string as an argument and packs the contents of arr into a binary sequence according to the directives in template (see Table 15, Directives for, on page 535). Directives A, a, and Z may be followed by a count, which gives the width of the resulting field. The remaining directives also may take a count, indicating the number of array elements to convert. If the count is an asterisk (*), all remaining array elements will be converted. The directives s S i I l L may be followed by an underscore (_) or bang (!) to use the underlying platform's native size for the specified type; otherwise, they use a platform-independent size. The directives s S i I l L q Q may be followed by a less than sign to signify little endian or greater than sign for big endian. Spaces are ignored in the template string. Comments starting with # to the next newline or end of string are also ignored.

On the other side, String#unpack takes a template string and decodes the string (which may contain binary data) according to the format string, returning an array of the extracted values. The directives are the same.